Software Requirements and System Design for Three Case Studies

Requirements, System Diagrams, Use Cases, Data Flow Diagrams and Solution Approach

Introduction

Good software starts with a clear understanding of what must be built and how the parts fit together. This assignment analyses three systems from different domains: (1) a **Document Management System** for an organisation, (2) a **Veterinary Surgical Management System** for an animal hospital, and (3) a **Blockchain and AI-based cyber-secure Intrusion Detection and Prevention System (IDS/IPS)** working as a component of SIEM and SOAR. Each is a realistic scenario with different users, risks and quality needs, from record keeping to patient safety to cyber defence.

Method followed for every scenario

1. **Requirements:** functional requirements (FR) describe the services the system must provide; non-functional requirements (NFR) describe quality levels such as security, performance and availability.
2. **System diagram:** a layered architecture showing the main components, data stores and external systems.
3. **Use case diagram:** UML actors and use cases, followed by a specification of one key use case.
4. **Data flow diagrams:** Level 0 (context) and Level 1 (main processes and data stores).
5. **Solution approach:** architecture style, technology, development process, security, testing and deployment.

Notation used in the diagrams

- **Architecture diagrams:** dashed boxes are layers, plain boxes are components, shaded boxes are external systems.
- **Use case diagrams:** stick figures are actors, ellipses are use cases, and the dashed rectangle is the system boundary.
- **Data flow diagrams:** shaded rectangles are external entities, ellipses are processes, open-ended rectangles are data stores (D1, D2, ...), and labelled arrows are data flows.

All diagrams are drawn in black and white so that the document prints clearly.

1. Document Management System (DMS)

1.1 Overview

A Document Management System lets an organisation create, store, find, share, review and retire documents securely from one place. It replaces scattered files and e-mail attachments with a controlled repository that keeps every version, enforces who may do what, and records every action for accountability.

1.2 Requirements

Functional requirements (what the system must do):

ID	Requirement
FR-1	Register users, authenticate them (password with optional multi-factor or single sign-on) and assign roles: Employee, Author, Approver, Administrator and Auditor.
FR-2	Upload or create documents in common formats (PDF, DOCX, XLSX, images) with mandatory metadata: title, author, category, tags and date.
FR-3	Organise documents in a folder and category hierarchy with naming and classification rules.
FR-4	Search by metadata and by full-text content, with filters, sorting and saved searches.
FR-5	Version control: check-in and check-out, automatic version numbers, version comparison and rollback.
FR-6	Role-based and document-level permissions (view, edit, delete, share), including share links that expire.
FR-7	Configurable review and approval workflow with deadlines, reminders and electronic sign-off.
FR-8	E-mail and in-app notifications for assigned tasks, approval results and documents about to expire.
FR-9	Tamper-resistant audit trail of every view, edit, download, share and deletion.
FR-10	Retention, archival and secure-disposal policies; scheduled backup and restore.
FR-11	Reports on usage, pending approvals and storage consumption.

Non-functional requirements (quality attributes and constraints):

ID	Category	Requirement
NFR-1	Security	Encryption at rest (AES-256) and in transit (TLS 1.3); least-privilege access; virus scan on upload.
NFR-2	Performance	Search results in under 2 seconds for one million documents with 200 concurrent users.
NFR-3	Availability	99.5% uptime; daily backups; recovery within 4 hours of a failure.
NFR-4	Scalability	Storage and users can grow by adding servers, without redesign.
NFR-5	Usability	Browser-based, responsive interface needing minimal training.
NFR-6	Maintainability	Modular services with documented APIs; configuration instead of code changes for workflows and retention rules.

1.3 System Diagram

The system is organised in layers; arrows show the main direction of requests and data between layers, users and external systems.

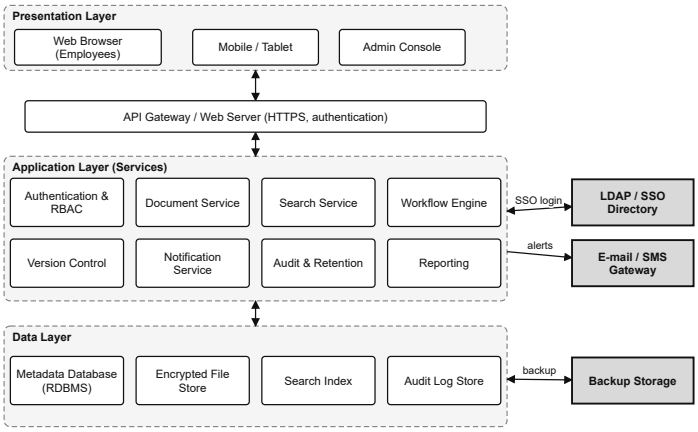


Figure 1.1: System (architecture) diagram

1.4 Use Case Diagram

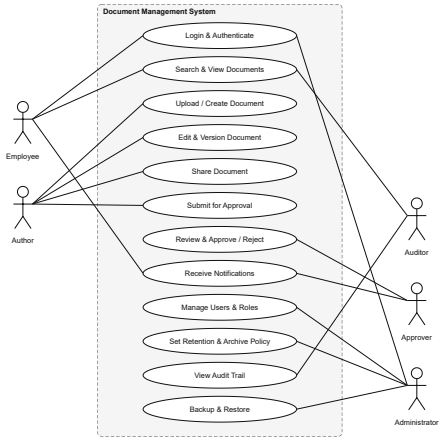


Figure 1.2: UML use case diagram (actors outside, use cases inside the system boundary)

Use case specification (key use case):

Use case	Submit Document for Approval
Actors	Author (primary), Approver (secondary)
Preconditions	Author is logged in and the document exists as a draft.
Main flow	1. Author selects the document and chooses Submit for approval. 2. System checks that mandatory metadata is complete. 3. Author selects the approver(s) and a deadline. 4. System creates the workflow task, locks the submitted version and notifies the approver. 5. Approver reviews the document and approves or rejects it with a comment. 6. System records the decision, updates the status and notifies the Author.
Alternative flows	Step 2: metadata missing, so the system asks the Author to complete it. Step 5: the Approver rejects, so the document returns to draft with comments. Deadline passed: reminder, then escalation to the approver's manager.
Postconditions	Document is Approved or Rejected; every step is recorded in the audit log.

1.5 Data Flow Diagrams

Level 0 (context diagram): the whole system is one process interacting with its external entities.

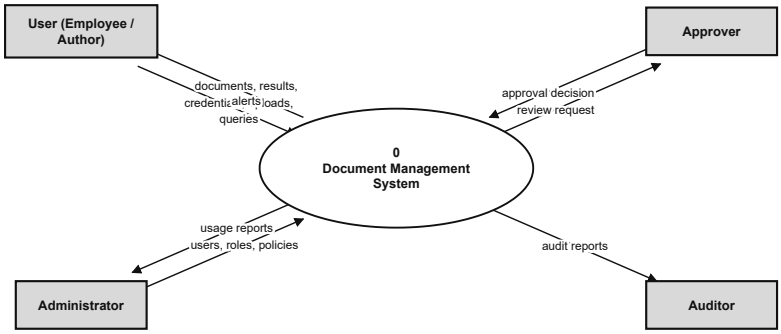


Figure 1.3: Data flow diagram, Level 0 (context)

Level 1: the system is decomposed into its main processes, data stores and flows.

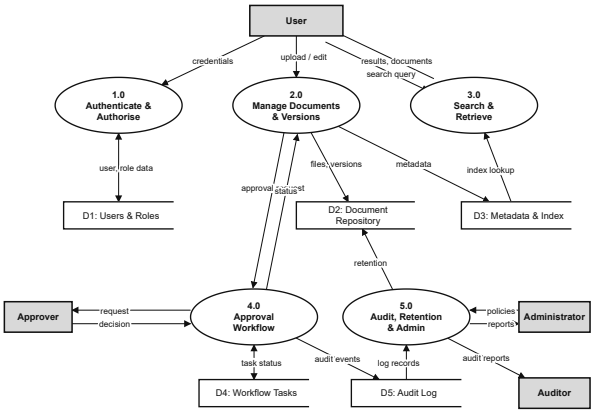


Figure 1.4: Data flow diagram, Level 1

- **1.0** checks credentials against D1 and returns the permissions of the session; **2.0** stores each new version in D2 and its metadata in D3.
- **3.0** looks up D3 and returns only the documents the user is permitted to see; **4.0** creates and tracks approval tasks in D4 and reports the status back to 2.0.
- **5.0** applies retention and archival rules to D2 and produces audit and usage reports from D5. For readability, the audit-event arrows from processes 1.0 to 3.0 into D5 are omitted; every process writes to D5.

1.6 Solution Approach

- **Architecture:** a three-tier web application (browser, application server with REST API, data layer) built from loosely coupled services so that search and workflow can scale independently.
- **Technology (example):** React front end; Java Spring Boot or Python Django back end; PostgreSQL for metadata; S3-compatible encrypted object storage for files; Elasticsearch/OpenSearch for full-text search; Keycloak or LDAP for single sign-on.
- **Development process:** Agile (Scrum) in two-week sprints. First release (MVP): login, upload, search and versioning. Second: workflow and notifications. Third: retention, reporting and analytics.
- **Security:** role-based access checked on every API call, encryption, antivirus scanning, and an append-only audit log; periodic access reviews.
- **Quality assurance:** unit and integration tests, load tests for search, security testing, and user acceptance testing with a pilot department.
- **Deployment and adoption:** Docker containers with CI/CD; import of legacy files through a metadata-mapping tool; user training. Risks (low adoption, migration errors, growth) are handled by training, a pilot migration and scalable storage.

2. Veterinary Surgical Management System (VSMS)

2.1 Overview

A Veterinary Surgical Management System supports a veterinary hospital through the whole surgical journey of an animal: registration, booking, pre-operative assessment, consent, theatre scheduling, anaesthesia and surgery records, post-operative recovery, medication, billing and follow-up. Its goals are patient safety, complete records and efficient use of surgeons, staff and theatre time.

2.2 Requirements

Functional requirements (what the system must do):

ID	Requirement
FR-1	Register pet owners and patients (species, breed, age, weight, microchip number, allergies and medical history).
FR-2	Book consultations and surgical appointments; prevent double-booking of surgeons, nurses, theatre and equipment.
FR-3	Pre-operative assessment: examination findings, laboratory and imaging results, risk classification and fasting instructions.
FR-4	Capture the owner's electronically signed informed consent and a cost estimate.
FR-5	Record anaesthesia and intra-operative data: drugs and doses, vital signs, procedure steps, staff, times and complications.
FR-6	Post-operative care: recovery monitoring, pain scoring, medication plan, discharge instructions and follow-up booking.
FR-7	Track instruments, sterilisation cycles and a controlled-drug log.
FR-8	Pharmacy and inventory: stock levels, reorder alerts, expiry tracking and dispensing linked to the patient record.
FR-9	Billing: estimates, itemised invoices, deposits, payments and insurance claims.
FR-10	Automatic SMS/e-mail reminders for appointments, fasting, medication and follow-up; owner portal to view progress.
FR-11	Reports: theatre utilisation, surgical outcomes, revenue and stock consumption.

Non-functional requirements (quality attributes and constraints):

ID	Category	Requirement
NFR-1	Security & privacy	Role-based access (surgeon, nurse, receptionist, accountant see only what they need); encrypted records; audit log of access.
NFR-2	Reliability	Must stay available during surgery: local fallback mode, automatic backups, recovery within 1 hour.
NFR-3	Performance	Patient record opens in under 2 seconds; at least 50 concurrent users.
NFR-4	Usability	Tablet-friendly with fast data entry in theatre (templates, drop-downs, minimal typing).
NFR-5	Interoperability	Integration with laboratory, imaging (DICOM) and payment systems through APIs.
NFR-6	Safety & compliance	Built-in surgical safety checklist, weight-based dose validation, and retention of records as required by veterinary regulations.

2.3 System Diagram

The system is organised in layers; arrows show the main direction of requests and data between layers, users and external systems.

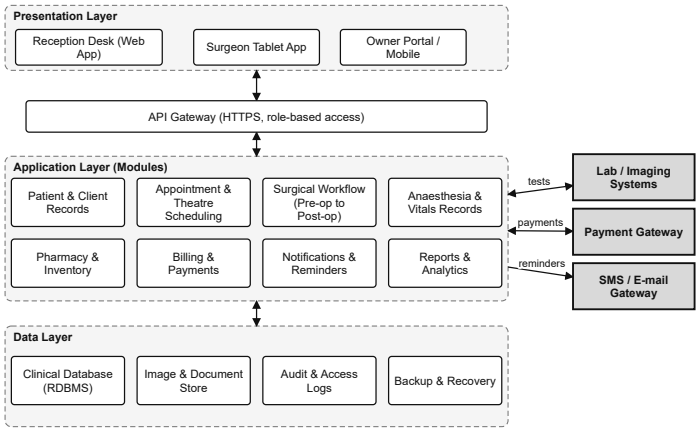


Figure 2.1: System (architecture) diagram

2.4 Use Case Diagram

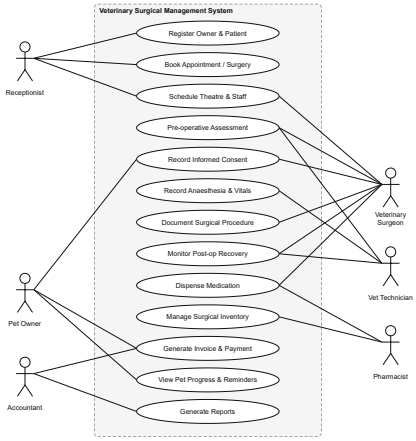


Figure 2.2: UML use case diagram (actors outside, use cases inside the system boundary)

Use case specification (key use case):

Use case	Perform Surgery and Record Intra-operative Data
Actors	Veterinary Surgeon, Vet Technician
Preconditions	Consent is signed, pre-operative assessment is complete, and theatre and staff are scheduled.
Main flow	1. Technician opens the case and confirms patient identity, weight and allergies using the safety checklist. 2. Technician records anaesthesia induction (drugs, doses, time). 3. Surgeon starts the procedure; the system time-stamps it. 4. Technician logs vital signs at set intervals. 5. Surgeon records steps, findings, implants, samples and any complication. 6. Surgeon ends the procedure; technician records the start of recovery. 7. System deducts used drugs and supplies from stock and posts the charges.
Alternative flows	Step 4: a vital sign out of range triggers an on-screen alert. Step 5: a complication triggers an emergency note and a notification to the owner.
Postconditions	Surgical record is saved, stock is updated, and the case moves to post-operative care.

2.5 Data Flow Diagrams

Level 0 (context diagram): the whole system is one process interacting with its external entities.

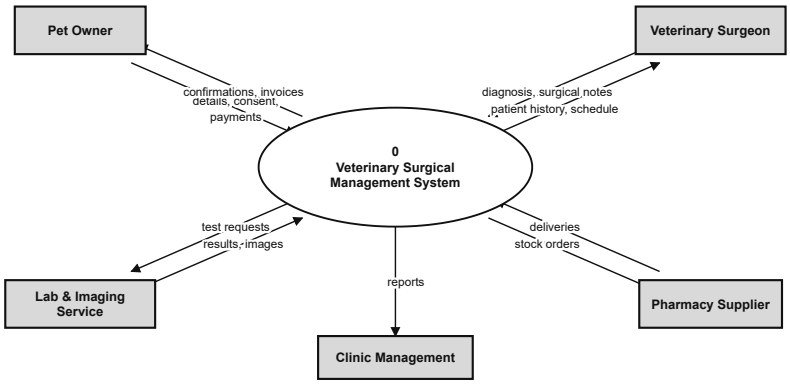


Figure 2.3: Data flow diagram, Level 0 (context)

Level 1: the system is decomposed into its main processes, data stores and flows.

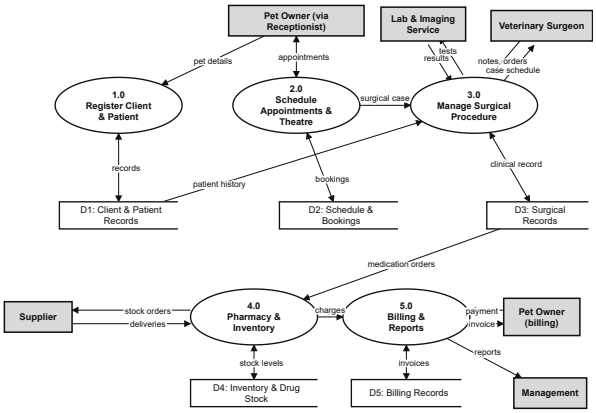


Figure 2.4: Data flow diagram, Level 1

- **1.0** creates the owner and patient record in D1; **2.0** checks resource availability and stores bookings in D2, then opens a surgical case for **3.0**.
- **3.0** reads the patient history from D1 and keeps the whole surgical record (pre-op, anaesthesia, intra-op, post-op) in D3; it exchanges test requests and results with the Lab & Imaging Service.
- **4.0** reads medication orders from D3, updates stock in D4 and reorders from the Supplier and records deliveries; **5.0** turns charges into invoices in D5, receives payments and reports to Management.

2.6 Solution Approach

- **Architecture:** a modular web application (patient, scheduling, surgical, pharmacy, billing and notification modules) behind one REST API, with a responsive progressive web app so that tablets work in theatre and reception.
- **Technology (example):** React or Angular front end; Node.js, Spring Boot or Django back end; PostgreSQL; object storage for X-ray images and consent forms; SMS/e-mail gateway; payment gateway integration.
- **Development process:** iterative and incremental with veterinary staff as product owners. Release 1: records and scheduling. Release 2: surgical workflow and anaesthesia charting. Release 3: pharmacy, billing, reminders and owner portal.
- **Patient safety by design:** surgical checklist embedded in the workflow, weight-based dose checks, mandatory fields, allergy warnings and controlled-drug logging.
- **Security and continuity:** role-based access, encryption, audit log, daily backups with an off-site copy, and a local fallback so that records stay available during surgery.
- **Testing and rollout:** scenario tests of a complete surgery from booking to discharge, user acceptance testing in the clinic, a pilot in one theatre, then full rollout with staff training.

3. Blockchain and AI-based Cyber-secure IDS/IPS within SIEM and SOAR

3.1 Overview

An Intrusion Detection System (IDS) monitors networks and hosts and raises alerts; an Intrusion Prevention System (IPS) sits in the traffic path and blocks attacks. A SIEM collects and correlates security events from all sources, and SOAR automates the response through playbooks. In the proposed design the IDS uses artificial intelligence to detect both known and unknown attacks, the IPS enforces the response, and a permissioned blockchain stores tamper-evident proof (hashes) of logs, alerts and response actions so that evidence cannot be silently altered.

3.2 Requirements

Functional requirements (what the system must do):

ID	Requirement
FR-1	Collect network traffic, host, application and cloud logs through sensors (TAP/SPAN), endpoint agents and syslog or API connectors.
FR-2	Normalise and enrich events (asset, user, geo-IP, threat intelligence) into a common schema and store them.
FR-3	IDS: detect known attacks by signatures and unknown or zero-day attacks by AI-based anomaly detection.
FR-4	Classify the attack type (denial of service, scan, brute force, malware, data exfiltration), assign severity and a risk score, and explain the decision.
FR-5	SIEM: correlate events across sources, raise alerts, and provide dashboards, search and log retention.
FR-6	IPS: automatically block, rate-limit or quarantine malicious traffic and hosts through the firewall, NGFW and endpoint agents.
FR-7	SOAR: run playbooks (enrichment, containment, ticketing, notification) with human approval for high-impact actions, and manage cases.
FR-8	Blockchain integrity: record hashes of logs, alerts and response actions on a permissioned ledger and verify that evidence has not been changed.
FR-9	Smart contracts: control who may change detection rules, approve response actions and share threat indicators; record each decision on the ledger.
FR-10	Model lifecycle: training, validation, scheduled retraining, drift monitoring and rollback.
FR-11	Compliance and forensic reports; multi-factor authentication and separation of roles (analyst, administrator, auditor).

Non-functional requirements (quality attributes and constraints):

ID	Category	Requirement
NFR-1	Performance	Inline IPS decision within milliseconds; ingest at least 100,000 events per second; alert within seconds of an attack.
NFR-2	Detection quality	Detection rate of at least 95% and false-positive rate of at most 2% on benchmark datasets and in testing.
NFR-3	Availability	Clustered, highly available components; documented fail-open or fail-closed policy for the IPS.
NFR-4	Integrity	Tamper-evident evidence; only hashes (not raw logs) are written on-chain, which also protects privacy.
NFR-5	Interoperability	Standard interfaces: syslog, CEF, REST, and STIX/TAXII for threat intelligence.
NFR-6	Compliance & scalability	Aligned with ISO/IEC 27001 and NIST guidance; horizontal scaling of collectors, models and ledger nodes.

3.3 System Diagram

The system is organised in layers; arrows show the main direction of requests and data between layers, users and external systems.

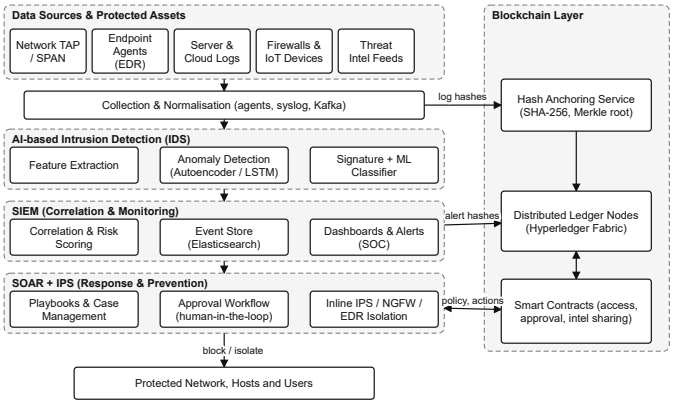


Figure 3.1: System (architecture) diagram

3.4 Use Case Diagram



Figure 3.2: UML use case diagram (actors outside, use cases inside the system boundary)

Use case specification (key use case):

Use case	Detect and Respond to an Intrusion
Actors	Monitored Systems (source), SOC Analyst, Incident Responder
Preconditions	Sensors and agents are active; AI models and playbooks are deployed.
Main flow	1. Sensors and agents send traffic and logs to the collector. 2. System normalises the events and writes the hash of each batch to the blockchain. 3. AI model scores the events; a score above the threshold creates an alert. 4. SIEM correlates the alert with other events and assigns a severity. 5. SOAR playbook enriches the alert with threat intelligence and proposes containment. 6. Analyst approves the action (automatic for low-risk actions allowed by policy). 7. IPS blocks the address or isolates the host. 8. System records the action and its hash on the ledger and updates the ticket.
Alternative flows	Step 3: score below threshold, so the event is only logged. Step 6: analyst rejects, so the case stays open for investigation. Step 7: enforcement fails, so the system retries and escalates.
Postconditions	Threat is contained; alert, response and evidence are tamper-evident; report is available.

3.5 Data Flow Diagrams

Level 0 (context diagram): the whole system is one process interacting with its external entities.

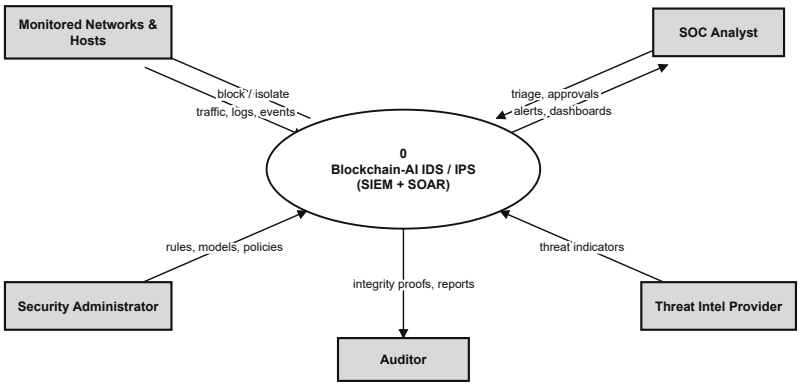


Figure 3.3: Data flow diagram, Level 0 (context)

Level 1: the system is decomposed into its main processes, data stores and flows.

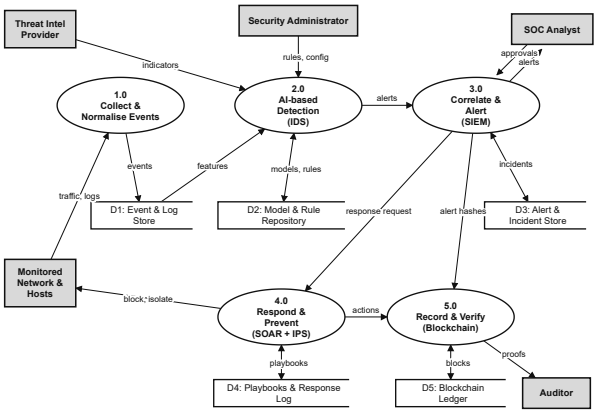


Figure 3.4: Data flow diagram, Level 1

- **1.0** normalises sensor and log data into D1; **2.0** extracts features from D1, applies the models and rules in D2 and the threat indicators from the provider, and produces scored alerts.
- **3.0** correlates alerts, stores incidents in D3 and shows them to the analyst, who approves actions; **4.0** executes the playbooks in D4 and tells the network or hosts to block or isolate.
- **5.0** writes hashes of alerts and actions to the ledger D5 and provides integrity proofs to the Auditor.

3.6 Solution Approach

- **Architecture:** a layered pipeline (collect, detect with AI, correlate in SIEM, respond with SOAR and IPS). Bulk data stays in an off-chain event store; only hashes and decisions go on-chain, which keeps the ledger fast and protects privacy.
- **Technology (example):** Zeek/Suricata sensors, Kafka streaming, Elasticsearch/Wazuh as SIEM, Python (scikit-learn, TensorFlow) for models, a SOAR platform such as Shuffle or Cortex, and Hyperledger Fabric as the permissioned blockchain with chaincode as smart contracts.
- **AI approach:** train on labelled datasets such as CICIDS2017 and UNSW-NB15 using flow features. Combine a supervised classifier (Random Forest or XGBoost) for known attacks with an autoencoder or LSTM for anomalies. Evaluate with precision, recall, F1 and ROC; retrain regularly; keep a human in the loop for high-impact actions.
- **Blockchain approach:** write the SHA-256 hash (Merkle root) of each log batch, each alert and each response action to the ledger; chaincode enforces approval and information-sharing rules. Verification recomputes the hash and compares it with the ledger entry.
- **Delivery in phases:** (1) sensors and SIEM baseline, (2) AI detection, (3) SOAR playbooks, (4) IPS in monitor-only mode and then in blocking mode, (5) blockchain anchoring. Test in a lab with simulated attacks (scans, brute force, denial of service) before production.
- **Risks and mitigation:** false positives that block legitimate traffic (monitor mode, thresholds, approval gates); adversarial inputs to the models (monitoring, retraining, ensemble methods); ledger overhead (batch hashing); sensor failure (high availability).

Conclusion

The three scenarios share one engineering path: requirements first, then structure (architecture and use cases), then behaviour (data flows), then a practical plan to build and verify the system. The Document Management System stresses access control and traceability; the Veterinary Surgical Management System stresses safety and reliability during surgery; and the Blockchain-AI IDS/IPS stresses speed, detection accuracy and tamper-evident evidence. Choosing a modular architecture, building in phases and testing early reduces risk in all three cases.